Capítulo 1

Aula 1

1.1 Notação assintótica

Uma boa referência para notação assintótica é o livro [Cormen et al., 2002], Seção 3.1, crescimento de funções, notação assintótica. Fica também como sugestão a leitura dos capítulos I e II deste livro.

Existem três principais notações assintóticas: O (lê-se ó) Θ (lê-se teta) e Ω (lê-se omega).

E do que se trata notação assintótica. Considere uma função $f(n) > 0$. Note que a função é sempre maior que zero. A teoria de notação assintótica só vale para funções positivas. Temos também uma segunda função $g(n) > 0$. Temos também que $n > 0$. Existem estas duas funções positivas.

Mas que funções são estas? Em geral, a função $f(n)$ é o tempo de execução do algoritmo ou o número de operações do algoritmo, onde n é, em geral, o tamanho da instância. Por exemplo, n pode representar o número de elementos em um vetor e $f(n)$ o tempo para se ordenar o vetor.

Se eu tiver um grafo, n pode ser o número de vértices ou o número de arestas do grafo.

Temos que $f(n) = O(g(n))$ se e somente se existirem duas constantes positivas c e n_0 tal que $0 \leq f(n) \leq cg(n), \forall n > n_0$.

Podemos usar tanto $f(n) \in O(g(n))$ quanto $f(n) = O(g(n))$. Pela definição, para que $f(n)$ seja $O(g(n))$, podemos desconsiderar valores pequenos de n. Podemos iniciar em um valor grande, porém constante n_0. Somente a partir de n_0 é que $f(n)$ precisa ser limitada superiormente por $cg(n)$. Outra coisa que podemos fazer é aumentar a função $g(n)$ tanto quanto for necessário, multiplicando-a por uma constante c.

Veja a Figura 1.1 para exemplificar a função O. A notação O diz respeito a um limitante superior.

Vamos ver um exemplo de como isso funciona. Seja $f(n) = 10n^2 + 5n + 5$ e $g(n) = n^2$. Mostre que $10n^2 + 5n + 5 = O(n^2)$.

Vou fazer uma tentativa e usar a constante igual a $c = 10$.

$$10n^2 + 5n + 5 \quad \leq \quad 10n^2, \forall n > 0 \ (Falso) \tag{1.1}$$

A inequação acima está errada, pois o lado esquerdo é maior que o lado direito. Suponha agora que $n > 1$. Nestas circustância $n \leq n^2$ e $1 \leq n^2$.

$$10n^2 + 5n + 5 \quad \leq \quad 10n^2 + 5n^2 + 5n^2, \forall n > 1 \tag{1.2}$$

$$10n^2 + 5n + 5 \quad \leq \quad 20n^2, \forall n > 1. \tag{1.3}$$

Neste momento termina a prova. Temos que $10n^2 + 5n + 5 = O(n^2)$ pois existem constantes positivas

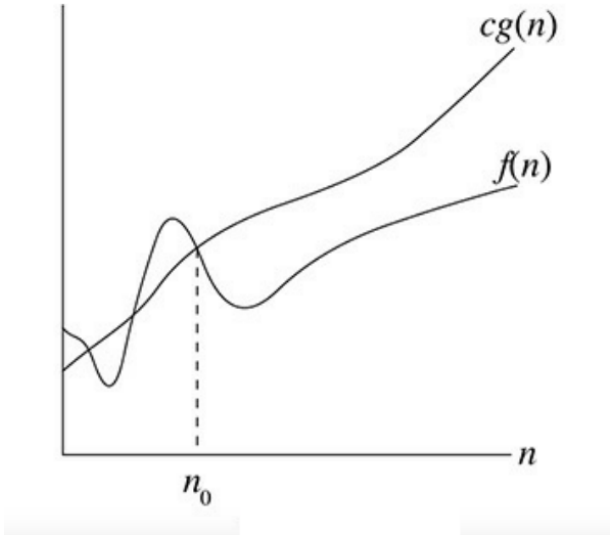

Figura 1.1: Notação assintótica $f(n) = O(g(n))$. Fonte: [Cormen et al., 2002]

$c = 20$ e $n_0 = 1$ tal que $10n^2 + 5n + 5 \leq 20n^2$, para todo $n > 1$. Note que eu poderia usar outras constantes como $c = 30$ o $c = 40$ e poderia usar $n > 10$ ou $n > 100$ que a a afirmação continuaria verdadeira.

A função O diz o seguinte. Eu posso multiplicar minha função n^2 por uma constante. Será que eu consigo fazer n^2 passar $10n^2 + 5n + 5$? Eu também posso desprezar valores pequenos de n. No caso, todo $n \leq 1$ é desprezado.

O que esta notação assintótica quer dizer? Todas as funções a seguir também são $O(n^2)$:

- $100n^2$
- $5n^2 + 5n$
- $5n^2 + \log n$

Eu agrupo várias funções e digo que todas elas são $O(n^2)$. todas estas funções são limitadas superiormente por n^2. No caso $5n^2 + 5n$ eu olho apenas para o termo mais importante para valores grandes de n, que é o $5n^2$. Para valores grande de n, $5n^2$ vai ser muito maior que $5n$ e assim o $5n$ pode ser desprezado. Eu também posso desprezar constantes multiplicativas, e olhar apenas para o n^2. Notação assintótica é uma teoria para grandes números. Ao ignorar-se as constantes, $5n^2$ e $1000000n^2$ tem a mesma complexidade $O(n^2)$.

Sendo assim temos classes de algoritmos: algoritmos $O(n^2)$, que são chamados quadráticos; algoritmos $O(n)$ que são chamados lineares; algoritmos $O(\log n)$ que são logarítmicos e assim por diante.

É claro que $n = O(n^2)$ mas neste caso temos folga. Temos que $n = O(n)$, e nesse caso temos uma notação justa. Temos também, de maneira imediata que $n^2 = O(n^3)$, $n^2 = O(n^4)$ e assim por diante. Nestes casos também temos folga.

Quando é que a notação O dá errado? Vamos tentar provar que $n = O(\sqrt{n})$, ou seja, que n é limitada superiormente por $\sqrt{n}$. De fato não é, mas vamos tentar provar e ver o que acontece.

Vamos considerar a função $\sqrt{n}$. Vamos multiplicar $\sqrt{n}$ por uma constante bem expressiva, como 10^6. Sera que $n \leq 10^6\sqrt{n}$ para todo $n \geq n_0$?

Não estamos interessados em valores pequenos de n. Podemos desconsiderar $n < n_0$. Independente da constante que eu use, n sempre vai ultrapassar $\sqrt{n}$. Mas vamos tentar.

$$n \leq c\sqrt{n}, n \geq n_0 \tag{1.4}$$
$$n^1 \leq cn^{0.5}, n \geq n_0 \tag{1.5}$$
$$n \leq cn^{0.5}, n \geq n_0 \tag{1.6}$$
$$n^{0.5} \leq c, n \geq n_0 \tag{1.7}$$
$$\sqrt{n} \leq c, n \geq n_0 \tag{1.8}$$

Após alguns cálculos chegamos a uma contradição. Nossa constante c precisa ser maior que $\sqrt{n}$ e este valor vai para infinito. Não existe constante nestas condições. Sendo assim, $n \neq O(\sqrt{n})$.

Algo análogo acontece ao se tentar provar que $n^3 = O(n^2)$. Temos que, para ser O temos que $n^3 \leq cn^2, \forall n > n_0$. Chegamos a contradição que $c \geq n$. Qualquer constante oferecida, existe um n que inviabiliza esta constante.

Considere o código abaixo:

```
#include <stdio.h>
int main(){
    int n=100;
    int x = 0;
    for(int i=0;i<n;i++){
        x = x+3;
    }
}
```

Neste caso eu pergunto quantas operações são gastas pelo algoritmo em função de n. Suponha que o valor de n possa ser mudado. Temos o seguinte:

1. declaração e atribuição de n: 2 operações;
2. declaração e atribuição de x: 2 operações;
3. declaração e atribuição de i: 2 operações;
4. comparação $i < n$: n+1 operações;
5. $i++$: n operações;
6. $x = x + 3$: n operações;

O número total de operações é $T(n) = 3n+7$. Nestes casos não estamos interessados no número exato de operações. O que é importante é que o número de operações é $T(n) = O(n)$. Ou seja, o algoritmo é linear.

A outra notação assintótica que é de nosso interesse é a Θ (lê-se teta).

Temos que $f(n) = \Theta(g(n))$ se e somente se existirem constantes positivas c_1, c_2 e n_0 tais que $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ para todo $n > n_0$.

Novamente não estou interessado em valores pequenos. A partir de um certo ponto a função $f(n)$ está numa espécie de sanduíche. Temos $g(n)$ multiplicada por uma constante c_1 que derruba $g(n)$ e a deixa sempre menor que $f(n)$ para $n > n_0$ e c_2 que levanta a função $g(n)$ e a deixa sempre maior que $f(n)$ para $n > n_0$. $f(n)$ fica no meio de $g(n)$ a partir de um certo ponto e até o infinito. Veja Figura 1.2.

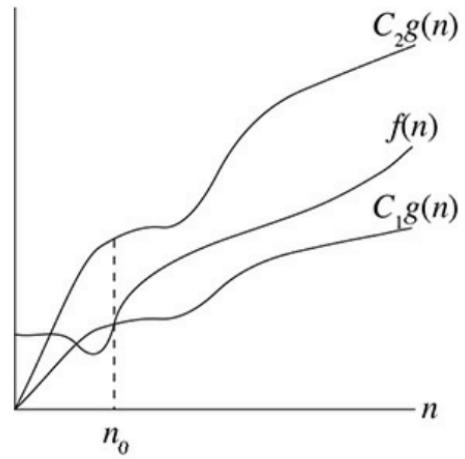

Figura 1.2: Notação assintótica $f(n) = \Theta(g(n))$. Fonte: [Cormen et al., 2002]

Seja $f(n) = 100n^2 + 100n + 100$. Eu gostaria de mostrar que $f(n) = \Theta(n^2)$. Vou assumir que $n > 1$. Nestas circunstâncias eu tenho que:

$$100n^2 + 100n + 100 \leq 100n^2 + 100n^2 + 100n^2, \forall n > 1 \tag{1.9}$$

$$100n^2 + 100n + 100 \leq 300n^2, \forall n > 1 \tag{1.10}$$

$$\tag{1.11}$$

Esta parte é idêntica aos cálculos feitos para O. Agora temos que mostrar o outro lado:

$$n^2 \leq 100n^2 + 100n^2 + 100n^2, \forall n > 1 \tag{1.12}$$

Esta parte também é verdadeira.

Sendo assim, $f(n) = \Theta(n^2)$ pois existem constantes positivas $c_1 = 1$, $c_2 = 300$ e $n_0 = 1$ tal que $0 \leq n^2 \leq f(n) \leq 300n^2, \forall n \geq 1$.

Exercícios:

Questão 1) Mostre que $100n^2 + 200n^2$ é $\Theta(n^2)$.

Questão 2) Mostre que $n \log n = O(n^2)$.

Questão 3) Mostre que, dado um polinômio de grau k, com o seguinte formato $P(x) = a_0x^0 + a_1x^2 + ... + a_kx^k$ onde os valores a_i são constantes. Mostre que $P(x) = \Theta(x^k)$.

Questão 4) Justifique ou refute as seguintes afirmativas:

1. Se $f(n) = O(g(n))$ então $f(n) = \Theta(g(n))$.
2. Se $f(n) = \Theta(g(n))$ então $f(n) = O(g(n))$.
3. Se $f(n) = \Theta(g(n))$ então $g(n) = \Theta(f(n))$.

Questão 5) Responda "sim" ou "não". Justifique sua resposta com uma prova matemática.

- $10n^2 + 10n - 100 = O(n^2)$?

- $n^2 = O(10n^2 + 10n - 100)$?
- $100n^3 = O(n^4)$?
- $n^4 = O(n^3)$?
- $2^n = O(3^n)$?
- $3^n = O(2^n)$?
- $\log_{10}(n) = O(\log_2 n)$?
- $10n^2 + 10n - 100 = \Theta(n^2)$?
- $n^2 = \Theta(10n^2 + 10n - 100)$?
- $100n^3 = \Theta(n^4)$?
- $2^n = \Theta(3^n)$?
- $3^n = \Theta(2^n)$?
- $\log_{10}(n) = \Theta(\log_2 n)$?

Capítulo 2

Aula 2

Nesta aula vamos dar continuidade ao tema notação assintótica.

2.1 Notação assintótica e tempo de execução

Continuando, temo que existem cinco notações assintóticas: $O, o, \Theta, \Omega, \omega$. Lê-se ozão, ozinho, teta, omegazão e omegazinho. No nosso curso não iremos trabalhar com o e ω. As mais importantes são O e Θ.

As vezes dizemos que um pesquisador é mais focado no $O(n)$. Isso significa que tal pesquisador está mais preocupado com a complexidade do algoritmo do que em detalhes de eficiência e programação.

A complexidade de um programa é descrito por meio de notação assintótica. Por exemplo, um algoritmo tem complexidade $O(n^2)$ ou tem complexidade $O(n^3)$. O programa de ordenação, por exemplo, pode ter complexidade $O(n \lg n)$.

Note que $\log n$ significa $\log_{10}$ enquanto $\lg n$ significa $\log_2$. Em termos de notação assintótica, não faz diferença usar $\log n$ e $\lg n$ pois a diferença entre eles é apenas uma constante. Ou seja, $\log n = \Theta(\lg n)$.

Temos também complexidades exponenciais. Um algoritmo pode ser $\Theta(2^n)$ ou $\Theta(n!)$. Algoritmos exponenciais são os piores e tendem a executar em tempo hábil apenas para valores muito pequenos de n.

Todo programa está associado a um tempo e este tempo é descrito por meio de notação assintótica. Veja o código abaixo:

```
for(int i=0;i<n;i++){
    for(int j=0;j<n;j++){
        v[i] = i+j;
    }
}
```

Quantas operações o programa gasta para fazer dois fors encadeados? O laço mais interno vai gastar $laco_j(n) = 1 + (n+1) + n + n = 3n + 2$ pois

- $int\ j = 0$: 1 operação ;
- $j < n$: $n + 1$ operações;
- $j++$: n operações;
- $v[i] = i + j$: n operações.

O laço mais externo vai gastar algo semelhante:

$$T(n) = 1 + (n+1) + n + n(laco_j(n)) \tag{2.1}$$
$$T(n) = 2n + 2 + n(3n+2) \tag{2.2}$$
$$T(n) = 3n^2 + 4n + 2 \tag{2.3}$$

Não estamos preocupados com valores pequenos de n. Gostariamos de saber $T(n)$ para $n = 1.000$ ou $n = 1.000.000$. Quando n é pequeno, todo programa é rápido.

Para $n = 1000$, $T(n) = 3 * 1000 * 1000 + 4 * 1000 + 2$. Perto de 3.000.000 tanto o 4.000 quanto o 2 são muito pequenos. Podemos assumir que $T(n) = 3 * 1000 * 1000$ inserindo um erro de apenas 0.1%. Podemos jogar fora inclusive a constante 3 em $3n^2$ e deixar apenas $T(n) = \Theta(n^2)$.

A notação assimtótica é uma maneira de simplificar o tempo de execução ou número de operações de algoritmos, jogando fora parte da informação e simplificando o resultado. No fim dizemos apenas que o tempo de execução do algoritmo é $\Theta(n^2)$.

Vamos dizer agora que o um programa gasta tempo $T_1(n) = n10^{-6}s$. A constante 10^{-6} varia de computador para computador. Um outro algoritmo gasta $T_2(n) = n^2 10^{-6}s$. Se n for igual a 1 milhão, quanto tempo gastam os programas 1 e 2?

Para o programa 1 temos:

$$T_1(n) = n10^{-6}s \tag{2.4}$$
$$T_1(10^6) = 10^6 10^{-6}s \tag{2.5}$$
$$= 1s \tag{2.6}$$

Já para o programa 2 temos:

$$T_2(n) = n^2 10^{-6}s \tag{2.7}$$
$$T_1(10^6) = (10^6)^2 10^{-6}s \tag{2.8}$$
$$= 1.000.000s \tag{2.9}$$
$$\approx 11dias. \tag{2.10}$$

Enquanto o primeiro programa gasta 1s, o segundo programa gasta 1 milhão de segundos. Se o programador cometeu um erro e, ao invés de fazer um programa $\Theta(n)$ fez um programa $\Theta(n^2)$, o programa fica 1 milhão de vezes mais lento para $n = 1.000.000$. Para exemplificar, vamos analisar uma questão que foi deixada como exercício, o cálculo da média e do desvio padrão.

O código abaixo está correto, porém intencionalmente ineficiente:

```
double media(double v[], int n){
    double sum=0;
    for(int i=0;i<n;i++){
        sum+=v[i];
    }
    return sum/n;
}
double dvp(double v[],int n){
    double sum=0,aux;
    for(int i=0;i<n;i++){
```

```
        aux=v[i]-media(v,n);
        sum+=aux*aux;
    }
    sum = sum /(n-1);
    return sqrt(sum);
}
```

O calculo da média é $\Theta(n)$. O cálculo de desvio padrão é um for que repete n vezes alguns cálculos, entre eles o cálculo da média. Como o calculo da média é repetido n vezes, a complexidade final fica em $\Theta(n^2)$.

Para deixar o cálculo do desvio padrão $\Theta(n)$ basta salvar a média em uma variável que é calculada uma única vez, confome mostrado abaixo:

```

double dvp(double v[],int n){
    double soma=0,aux,med;
    med = media(v,n);
    for(int i=0;i<n;i++){
        aux=v[i]-med;
        sum+=aux*aux;
    }
    sum = sum /(n-1);
    return sqrt(sum);
}
```

Este tipo de erro é bastante comum, principalmente para programadores iniciantes. Programadores que não dominam estrutura de dados também estão propensos criar versões não otimizadas de algoritmos.

A partir da análise destes programas fica claro que não interessa muito a constante. Se um programa gasta 1s e outro gasta 1 milhão de segundos, mesmo que a constante do mais lento mude de 10^{-6} para 3×10^{-6} a diferença continua muito grande. A diferença entre o $\Theta(n)$ e o $\Theta(n^2)$ é tão grande que a relevância das constantes de tempo se torna pequena.

Já para valores pequenos de n, como $n = 10$, a diferença de tempos torna-se insignificante, a não ser que o algoritmo seja executado muitas vezes.

Em estrutura de dados é desejável sempre reduzir a complexidade do algoritmo. Transformar um algoritmo $\Theta(n^2)$ em outro $\Theta(n \log n)$. Transformar um algoritmo $\Theta(n)$ em outro $\Theta(\log n)$.

Uma terceira notação assintótica que ainda não foi citada é a Ω. Trata-se de um limitante inferior para uma função.

Temos que $f(n) = \Omega(g(n))$ se e somente se existirem constante positivas c e n_0 tais que $0 \leq cg(n) \leq f(n)$ para todo $n > n_0$. Neste caso, $f(n)$ é limitada inferiormente por $g(n)$. Veja o exercício abaixo:

Questão 6) Mostre que $g(n) = \Theta(g(n))$ se e somente se $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$.

Temos que $n^2 = \Omega(n^2)$ e também que $n^2 = \Omega(n)$, $n^2 = \Omega(\sqrt{n})$, $n^2 = \Omega(\log n)$ e assim por diante.

Vamos agora ver como se aplica a notação assintótica em desevolvimento de algoritmos. Considere o código abaixo:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
4 //tamanho maximo do vetor.
5 #define MAX 1000000
6 /**
7  Gera um vetor de numeros aleatorios de tamanho n. O valor
8  de cada elemento esta entre 0 e m-1.
9  Complexidade Teta(n).
10  */
11 void gera(int v[], int n , int m){
12     if(n>MAX){
13         printf("Vetor de tamanho insuficiente\n");
14         exit(1);
15     }
16     for(int i=0;i<n;i++){
17         v[i] = rand()%m;
18     }
19 }
20
21 /**
22  Imprime o vetor na tela.
23  Complexidade Teta(n).
24  */
25 void print(int v[], int n){
26     printf("[");
27     for(int i=0;i<n;i++){
28         printf("%d",v[i]);
29         if(i<n-1){
30             printf(",");
31         }
32     }
33     printf("]\n");
34 }
35 /**
36  Encontra a posicao do maior elemento do vetor.
37  Se houverem varios elementos nesta condicao,
38  devolve a posicao do primeiro maior elemento.
39  Complexidade Teta(n).
40  */
41 int maximo(int v[], int n){
42     int max = 0;
43     for(int i=1;i<n;i++){
44         if(v[i]>v[max]){
45             max = i;
46         }
47     }
48     return max;
```

```
49 }
50 /**
51  Ordena os elementos em ordem crescente via
52  ordenacao por selecao.
53  Teta(n^2)
54  */
55 void ordena(int v[], int n){
56     while(n>1){
57         //seleciona o maior elemento
58         int max = maximo(v,n);
59         //coloca na ultima posicao
60         int aux = v[n-1];
61         v[n-1] = v[max];
62         v[max]=aux;
63         //repete o processo no vetor
64         //desconsiderando o ultimo elemento
65         n--;
66     }
67 }
68 int main(){
69     clock_t start, end;
70     double cpu_time_used;
71     int v[MAX];
72     int n;
73     printf("Digite n:");
74     scanf("%d",&n);
75     gera(v,n,n);
76     if(n<200) print(v,n);
77     // Inicio da contagem de tempo
78     start = clock();
79     ordena(v,n);
80     // Fim da contagem de tempo
81     end = clock();
82     if(n<200) print(v,n);
83     printf("Ordenou\n");
84     // Calculo do tempo gasto em milisegundos
85     cpu_time_used = (1000.0*(double) (end - start)) / CLOCKS_PER_SEC;
86     printf("Tempo de execucao: %f milisegundos\n", cpu_time_used);
87
88
89 }
```

A saída deste programa será:

```
Digite n:10
[7,9,3,8,0,2,4,8,3,9]
```

```
[0,2,3,3,4,7,8,8,9,9]
Ordenou
Tempo de execucao: 0.003000 milisegundos
```

O primeiro vetor é o vetor aleatório e o segundo vetor é o vetor ordenado. Note que a função máximo é chamada n vezes. A primeira vez é passado n, em seguida $n-1$, depois $n-2$ a assim sucessivamente até chegar em 2. O número de operações é $n+(n-1)+(n-2)+...+3+2$. Esta soma é a soma de uma progressão aritmética (PA). A fórmula para calcular a soma de uma PA é $(a_1+a_n)n/2$. Em nosso caso temos $\frac{(n+2)(n-1)}{2} = \frac{n^2}{2} + \frac{n}{2} - 1$. Ou seja, o número de operações feitas para calcular o máximo é $\Theta(n^2)$. Como as demais operações são $\Theta(n)$ o resultado final é $\Theta(n^2)$.

O código foi executado para $n = 100.000$ e o tempo resultante foi 10.723 ms. Ou seja, o programa gastou 10s para ordenar um vetor com 100.000 elementos. O tempo foi medido logo antes e logo depois da ordenação. Podemos dizer que, dentro de uma certa aproximação, que $T(n) = cte * n^2 ms$. Eu pego só a parte mais importante do polinômio e ignoro elementos proporcionais a n^1 e a n^0. Com estas informações eu posso calcular o valor da constante.

$$
\begin{aligned}
T(n) &= cte * n^2 ms && (2.11)\\
T(100000) &= cte * (100000)^2 ms = 10723 ms && (2.12)\\
cte * 10^{10} ms &= 10723 ms && (2.13)\\
cte &= 10723 * 10^{-10} && (2.14)\\
cte &= 1{,}0723 * 10^4 * 10^{-10} && (2.15)\\
cte &\approx 1,07 * 10^{-6} && (2.16)\\
& && (2.17)
\end{aligned}
$$

Com isso, eu tenho a fórmula $T(n) = n^2 * 10^{-6} ms$. Agora eu posso estimar o tempo gasto para $n = 200.000$.

$$
\begin{aligned}
T(200.000) &= 200.000^2 * 1,07 * 10^{-6} ms && (2.18)\\
&= (2 * 10^5)^2 * 1{,}07 * 10^{-6} ms && (2.19)\\
&= 4 * 10^{10} * 1{,}07 * 10^{-6} ms && (2.20)\\
&= 4 * 1{,}07 * 10^4 ms && (2.21)\\
&= 42800 ms && (2.22)\\
&= 42.8 s && (2.23)
\end{aligned}
$$

O tempo estimado é de 42.8s. Ao se executar o programa e medir o tempo, o resultado foi 42,734s. Note que a constante de tempo varia de computador para computador.

Questão 7) (Tempo de execução) Descubra a constante de tempo para o programa anterior (função ordena) no seu computador. Após isso, descubra quanto tempo vai demorar, em sua máquina, para ordenar um vetor com 1 milhão de elementos. E para 10 milhões de elementos? Crie uma planilha com 3 colunas. Na primeira o n, na segunda o $T(n)$ medido executando o programa em sua máquina, e na terceira, o $T(n)$ vindo da função, ou seja estimado a partir do calculo de $T(n) = cn^2$. Faça um gráfico com estas três colunas. Não foi passado os valores de n para fazer as medidas e o gráfico. É para escolher entre 5 e 10 valores bem distribuídos que rodem num tempo adequado.

A função de tempo se aproxima de uma parábola. Um erro muito comum é o aluno tentar utilizar uma regra de três numa função que não é linear. Por exemplo, se para um dado n o programa anterior gastou $1s$, quanto tempo levará para $10n$?

O raciocínio errado é

$$\frac{n}{10n} = \frac{1s}{x},$$

ou seja $x = 10s$.

O raciocínio correto é:

$$\begin{aligned} T(n) &= cte * n^2 && (2.24)\\ T(10n) &= cte * (10n)^2 && (2.25)\\ T(10n) &= 100cte * n^2 && (2.26)\\ T(10n) &= 100T(n) && (2.27)\\ & && (2.28) \end{aligned}$$

Ou seja, para $10n$ o tempo gasto será 100 vezes maior. Note que a constante *cte* muda de computador para computador. Ao se usar notação assintótica, as constantes *cte* são eliminadas.

Para finalizar, qual o tempo será gasto para n=10 milhões?

$$\begin{aligned} T(10^7) &= (10^7)^2 * 1,07 * 10^{-6}ms && (2.29)\\ &= 10^{14} * 1,07 * 10^{-6}ms && (2.30)\\ &= 1{,}07 * 10^8 ms && (2.31)\\ &= 107.000.000ms && (2.32)\\ &= 29{,}72h && (2.33) \end{aligned}$$

O programa irá demorar $29{,}72h$. Note que, neste caso, fica bem mais complicado executar o programa para testar se o tempo calculado está de acordo com o valor medido.

Questão 8) Descubra as complexidades das funções abaixo:

```
#include <stdio.h>


int f1(int n){
    int aux = 0;
    for(int i=0;i<n;i++){
        aux++;
    }
    return aux;
}


int f2(int n){
    int x=0;
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            x++;
```

```
18          }
19      }
20      return x;
21  }
22
23  int f3(int n){
24      int aux = 0;
25      for(int i=1;i<n;i=i*2){
26          aux++;
27      }
28      return aux;
29  }
30
31  int f4(int n){
32      int aux = 0;
33      for(int i=0;i<n;i++){
34          aux+=f1(i);
35      }
36      return aux;
37  }
38
39  int f5(int n){
40      int aux = 0;
41      for(int i=0;i<n;i++){
42          aux+=f1(n);
43          aux+=f2(n);
44      }
45      return aux;
46  }
47
48  int f6(int n){
49      int x=0;
50      for(int i=0;i<n;i+=100){
51          x+= 1;
52      }
53      return x;
54  }
55
56  int f7(int n){
57      if(n==0){
58          return 1;
59      }else{
60          int x = f7(n-1);
61          int y = f7(n-1);
62          return x+y;
```

```
    }
}

int main(int argc, const char * argv[]) {
    int n = 10;

    printf("f1=%d\n",f1(n));
    printf("f2=%d\n",f2(n));
    printf("f3=%d\n",f3(n));
    printf("f4=%d\n",f4(n));
    printf("f5=%d\n",f5(n));
    printf("f6=%d\n",f6(n));
    printf("f7=%d\n",f7(n));

}
```

Questão 9) Proponha uma função com complexidade $\Theta(2^n)$ (a) usando *for/while* (b) sem usar *for/while*.

Considere o código da função $f3(n)$ do exercício anterior. Neste código, a cada iteração, o valor de i dobra. Sendo assim, em k iterações, o valor de i vai valer 2^k. Como o programa executa k iterações, ele é $\Theta(k)$. O programa pára quando i atingir n, ou seja, quando $2^k = n$.

$$\begin{aligned} 2^k &= n \\ \lg 2^k &= \lg n \\ k \lg 2 &= \lg n \\ k &= \lg n \end{aligned}$$

Note que $\lg(n) = \log_2(n)$. Veja que k representa o número de iterações. Neste caso, o algoritmo é $\Theta(\lg n)$.

Veja que $\lg n$ é um número que cresce muito lentamente. Se escolhermos $n = 2^{1000}$ que tem uma ordem de grandeza de 1 seguido de 301 zeros. Temos que $\lg 2^{1000} = 1000$. Isso quer dizer que um algoritmo $\Theta(\lg(n))$ tende a ser muito rápido.

Por outro lado, funções exponenciais tem o crescimento muito acelerado. O valor 10^n cresce de maneira muito rápida. Se um algoritmo for $T(n) = 10^n \times 10^{-6}s$, para valores pequenos de n teremos um tempo muito grande.

Suponha que $T(n)$ seja 1 século. Neste caso, $T(n) = 100 \times 365 \times 24 \times 3600 = 3.153.600.000s$.

$$\begin{aligned} 10^n \times 10^{-6}s &= 3.153.600.000s \qquad (2.34) \\ 10^n &= 3.153.600.000 \times 10^6 \\ n &= \lg_{10}(3.153.600.000 \times 10^6) \\ n &= 15{,}49 \end{aligned}$$

Isso significa que para $n = 15$ o valor ainda é menor que 1 século, mas para $n = 16$ o tempo já supera um

século. O programa precisa executar por mais de 1 século para resolver um problema de tamanho apenas $n = 16$.

Além dos algoritmo exponenciais e logarítmicos, existem os algoritmos polinomiais. Eles estão no meio. São melhores que os exponenciais e piores que o logarítmicos. Eles são algoritmo $\Theta(n^k)$ para um k constante. Para $k = 1$ temos os algoritmos lineares, ou seja $\Theta(n)$. Para $k = 2$ temos os algoritmos quadráticos, ou seja $\Theta(n^2)$. Para $k = 3$ temos os algoritmos cúbicos, ou seja $\Theta(n^3)$.

Quanto menor o expoente do polinômio, melhor o algoritmo. Um algoritmo $\Theta(n)$ é melhor que um algoritmo $\Theta(n^2)$ que é melhor que um algoritmo $\Theta(n^3)$ e assim por diante.

Questão 10) Suponha que $T(n) = n^3 cte\ s$. Se $T(10.000) = 1000s$, qual a constante de tempo? Qual o tempo $T(10^6)$?

Capítulo 3

Aula 3

Esta aula trata de tempo de execução de algoritmos e tipo abstrato de dados.

3.1 Tempo de execução de algoritmos

Questão 11) (tempo de execução) Este exercício foi extraído de [Cormen et al., 2002], Problema 1.1. Para cada função $f(n)$ e cada tempo t, determine o maior tamanho de n de um problema que pode ser resolvido no tempo t, supondo que o algoritmo para resolver o problema demore $f(n)$ microsegundos segundos.

$f(n)$	1 segundo	1 minuto	1 hora	1 dia	1 mês	1 ano	1 século
$\lg n$							
$\sqrt{n}$							
n							
$n \lg n$							
n^2							
n^3							
2^n							
3^n							
$n!$							

Vou responder alguns exemplos e deixar outros como exercício. Suponha que $f(n) = \lg n$ e que o tempo é $1s$. Para um dado n, o algoritmo gasta $\lg n * 10^{-6}s$. Temos então que $\lg n * 10^{-6}s = 1s$,

$$\lg n * 10^{-6}s = 1s \tag{3.1}$$

$$\lg n = 10^6 \tag{3.2}$$

$$n = 2^{1.000.000} \tag{3.3}$$

$$= (10^{0,3010299\ldots})^{1.000.000} \tag{3.4}$$

$$= 10^{301029,9\ldots} \tag{3.5}$$

O maior número calculável em $1s$ é n igual a $10^{301029,9\ldots}$ ou seja, o número 1 seguido de mais de 300.000 zeros. Isso indica que um algoritmo regido pela função $\lg n$ é extremamente rápido, mesmo para valores muito grandes de n.

Questão 12) Suponha que $f(n) = \log n$ log na base 10. Qual o valor de n em 1s?

Vamos supor agora que $f(n) = n^2$ e que o tempo é 1 dia. Neste caso temos que

$$n^2 * 10^{-6}s = 24 * 3600s \tag{3.6}$$

$$
\begin{aligned}
n^2 * 10^{-6}s &= 86400s && (3.7)\\
n^2 &= 86400 * 10^6 && (3.8)\\
n &= \sqrt{86400 * 10^6} && (3.9)\\
&= 293938{,}76 && (3.10)
\end{aligned}
$$

Ou seja, o maior n que executa em um dia é 293.938.

Questão 13) Suponha agora um computador 10x mais rápido. Qual o valor o valor de n pode ser executado em 1 dia?

Suponha agora que $f(n) = 2^n$ e que o tempo é 1 século:

$$
\begin{aligned}
2^n * 10^{-6}s &= 100 * 365,25 * 24 * 3600s && (3.11)\\
2^n &= 100 * 365,25 * 24 * 3600 * 10^6 && (3.12)\\
\lg 2^n &= \lg(100 * 365,25 * 24 * 3600 * 10^6) && (3.13)\\
n &= 51.48 && (3.14)\\
& && (3.15)
\end{aligned}
$$

Nesse caso temos o oposto da função lg. A função 2^n cresce de maneira tão grande que, em um século, seria possível executar uma instância de tamanho apenas 51.

Questão 14) Suponha agora que $f(n) = 3^n$. Qual o valor de n pode ser executado em um século?

Suponha que $f(n)$ seja n^3 e que $T(n) = 1$ mês. Qual o maior valor de n?

A formula diz o seguinte:

$$T(n) = f(n)10^{-6}s$$

O tempo na fórmula está em convertido de microsegundos para segundos. Note que 1 mês vale $30 \times 24 \times 3600 = 2.592.000$ s. Veja que um mês tem 30 dias, um dia tem 24h e uma hora tem 3600s.

Sendo assim,

$$
\begin{aligned}
2.592.000s &= n^3 10^{-6}s && (3.16)\\
n^3 10^{-6} &= 2.592.000 \\
n^3 &= 2.592.000 \times 10^6 \\
n &= \left(2.592.000 \times 10^6\right)^{1/3} \\
n &= 13736
\end{aligned}
$$

A interpretação deste resultado é que, em um computador com constante de tempo $10^{-6}s$, em um mês de processamento, é possível resolver um problema com n = 13.736 usando um algoritmo regido por $T(n) = n^3 10^{-6}s$.

Questão 15) Suponha agora um computador 1000x mais rápido. Qual o valor de n pode pode ser executado em 1 mês?

Os demais campos da tabela são deixados como exercícios.

Questão 16) (tempo de execução) Suponha que um algoritmo demora $T(n) = 10^n 10^{-6}s$. Qual o maior valor de n que pode ser executado em 1 milhão de anos?

Para resolver este problema, basta igualar $10^n 10^{-6}s$ ao número de segundos de 1 milhão de anos.

$$
\begin{aligned}
10^n * 10^{-6}s &= 10^6 * 365,25 * 24 * 3600s && (3.17)\\
10^n &= 10^6 * 365,25 * 24 * 3600 * 10^6 && (3.18)
\end{aligned}
$$

$$\log 10^n = \log(365,25 * 24 * 3600 * 10^12) \tag{3.19}$$
$$n = 19.49 \tag{3.20}$$

No caso, o maior n inteiro é 19. Ou seja, em um milhão de anos será possível resolver um problema com n valendo apenas 19.

Questão 17) (Complexidadde) Suponha que eu tenho duas matrizes $B_{n\times n}$ e $C_{n\times n}$ e desejo calcular $A_{n\times n} = B_{n\times n} + C_{n\times n}$. Qual a complexidade?

Resposta: Para percorrer uma matriz quadrada, é necessário fazer um *for* varrendo as linhas e, para cada linha, fazer um *for* percorrendo as colunas. Isso gera dois *fors* encadeados. A complexidade é $\Theta(n^2)$.

Suponha que você está analisando um código grande, composto por centenas de linhas. Suponha que existem trechos $\Theta(n)$, $\Theta(n \lg n)$, $\Theta(1)$ e apenas um pequeno trecho, de poucas linhas, $\Theta(n^2)$. Este pequeno trecho irá dominar o tempo de execução do código inteiro. Ele vai ser o gargalo do seu programa. Para localizar um gargalo, é possível analisar os códigos manualmente ou usar uma ferramenta chamada $profiler$ que detecta trechos de códigos mais lentos.

Questão 18) (Complexidadde) Suponha agora que eu tenho duas matrizes $B_{n\times n}$ e $C_{n\times n}$ e desejo multiplicar $A_{n\times n} = B_{n\times n}C_{n\times n}$. Qual a complexidade?

Resposta: A resposta para este problema é mais complexa. Suponha que eu desejo calcular $a_{ij} \in A_{n\times n}$. Para se obter o elemento a_{ij} é necessário fazer o produto interno da linha i de B pela coluna j de C.

$$a_{ij} = \sum_{k=1}^{n} b_{ik}c_{kj}.$$

Desta maneira obtemos a_{ij} em $\Theta(n)$. Como existem n^2 a_{ij} distindos, o tempo final é $\Theta(n^3)$.

Podemos propor algoritmos mais ineficientes para multiplicar duas matrizes, como $\Theta(n^4)$, mas isso não faz muito sentido. A pergunta mais importante é: existem algoritmo ainda melhores que $\Theta(n^3)$?

A resposta é sim. O algoritmo proposto em [**?**] obteve uma $O(n^{\log_2 7}) \approx O(n^{2.8704})$.

O melhor algoritmo para multiplicação de matrizes foi obtido em [**?**] que obteve complexidade $(n^{2.37286})$.

Não sabemos, na data atual, se é possível encontrar algoritmos para multiplicar matrizes ainda melhores. O limite $O(n^2)$ não pode ser baixado, mas explorar valores entre n^2 e $n^{2.37286}$ ainda é um campo aberto para pesquisas.

Questão 19) (Tempo de execução). Suponha um algoritmo para multiplicar matrizes com tempo de execução $T(n) = n^3 10^{-6}s$. Qual a maior matriz que pode ser multiplicada em $1h$.

Resposta:

$$T(n) = n^3 10^{-6}s = 3600s \tag{3.21}$$
$$n^3 = 3600 * 10^6 \tag{3.22}$$
$$n = \sqrt[3]{3600 * 10^6} \tag{3.23}$$
$$n = 1532{,}61. \tag{3.24}$$

Ou seja, o maior valor inteiro de n tal que duas matrizes $n \times n$ pode ser multiplicada em 1h pelo algoritmo proposto é 1.532. Veja que a matriz não é tão grande. Se eu tiver duas matrizes de dimensão 1532, isso irá levar uma hora.

Questão 20) (Tempo de execução) Suponha agora que o programa de multiplicar matrizes foi executado em um computador 10 vezes mais rápido. Qual o maior valor n pode ser resolvido neste computador?

Resposta: Se o computador é 10 vezes mais rápido, a constante de tempo irá mudar de 10^{-6} para 10^{-7}.

$$
\begin{aligned}
T(n) &= n^3 10^{-7} s = 3600s && (3.25)\\
n^3 &= 3600 * 10^7 && (3.26)\\
n &= \sqrt[3]{3600 * 10^7} && (3.27)\\
n &= 3301{,}92. && (3.28)
\end{aligned}
$$

Neste caso é interessante notar que o computador é 10 vezes mais rápido, mas a instância aumentou apenas $\sqrt[3]{10} = 2.15...$ vezes.

Referências Bibliográficas

[Cormen et al., 2002] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2002). Algoritmos: teoria e prática. *Editora Campus*, 2:296.

[Damas, 2006] Damas, L. (2006). *Linguagem C*. LTC.